



Universidad
de Alcalá

Estructura de datos PL1 2023- 24

AUTORES

Isaac Marval Hernández 51752280A

Pablo Brañas de la Sota 51533070Y

13/10/2023

Contenido

Especificación concreta de la interfaz de los TAD's implementados:	3
TAD's creados.....	3
Definición de las operaciones del TAD (Nombre, argumentos y retorno).	3
Solución adoptada: descripción de las dificultades encontradas.	4
Diseño de la relación entre las clases de los TAD implementados.....	4
Explicación de los métodos más destacados.	4
Explicación del comportamiento del programa.	8
Bibliografía	13

Detalles y Justificación de la implementación

Especificación concreta de la interfaz de los TAD's implementados:

TAD's creados

- ColaPedido
- ColaReserva
- PilaMesa
- NodoMesa
- NodoPedido
- NodoReserva

Definición de las operaciones del TAD (Nombre, argumentos y retorno)

- **ColaReserva:** Este código define una clase llamada Cola, que se utiliza para gestionar una cola de objetos Reserva. La clase incluye métodos para insertar y eliminar elementos de la cola, así como para mostrar su contenido. La estructura de la cola se implementa utilizando nodos enlazados, donde pnode es un puntero a un nodo. El constructor y destructor de la clase se encargan de inicializar y liberar los recursos necesarios.
- **ColaPedido:** la clase ColaPedido, que representa una cola de pedidos en un restaurante. Una cola es una estructura de datos donde los elementos se agregan al final y se eliminan desde el principio, siguiendo el principio "primero en entrar, primero en salir" (FIFO). La clase ColaPedido tiene un constructor y un destructor. Además, incluye métodos para insertar un pedido al final de la cola, eliminar un pedido del principio y mostrar la cola. La clase utiliza nodos de tipo NodoPedido para organizar los pedidos en la cola. Cada nodo contiene un objeto de tipo Pedido.
- **Pilamesa:** este código define la clase PilaMesa, que simula una pila de mesas en un restaurante. Tiene un constructor y un destructor, además de métodos para insertar y eliminar mesas en la pila, así como para mostrar su contenido. La clase utiliza la clase NodoMesa para gestionar los elementos de la pila. Además, contiene métodos para recorrer y editar la pila en función del número de personas en una reserva y la ubicación.
- **NodoMesa:** la clase llamada NodoMesa representa un nodo en una estructura de datos. Los nodos son como contenedores que pueden almacenar objetos, y se utilizan en estructuras enlazadas como pilas o colas. También establece relaciones de amistad con otras clases, como Pila y PilaMesa, lo que significa que estas clases pueden acceder a los miembros privados y protegidos de la clase NodoMesa. En la clase NodoMesa, hay un constructor que toma un objeto de tipo Mesa y un puntero al siguiente nodo en la estructura. También, se define un destructor para liberar recursos. En resumen, este código es fundamental para crear

nodos que almacenan objetos de tipo Mesa en una estructura de datos, y estas relaciones de amistad permiten su uso en estructuras más complejas, como pilas o colas. **NodoPedido:**

- **NodoReserva:** la clase Nodo, que se utiliza en estructuras de datos, con un constructor que almacena objetos Reserva y un destructor para liberar recursos. También establece la amistad con las clases Pila y Cola para acceder a miembros privados. Se utiliza una directiva de preprocesador para evitar la inclusión duplicada del código. La clase Nodo permite crear una lista enlazada de objetos Reserva.
- **NodoPedido:** la clase NodoPedido, que representa un nodo en una estructura de datos utilizada para gestionar pedidos en un restaurante. Los nodos son como contenedores que almacenan objetos de tipo Pedido. La clase NodoPedido tiene un constructor que toma un objeto Pedido y un puntero al siguiente nodo en la estructura. También se define un destructor para liberar recursos cuando se elimina un nodo. Además, se establecen relaciones de amistad con la clase ColaPedido, lo que permite que esta clase acceda a los miembros privados y protegidos de la clase NodoPedido.

Solución adoptada: descripción de las dificultades encontradas.

PUNTEROS

Gestion de memoria, crear las todo en la memoria dinámica usando new y saber cuando utilizar el delete para borrar.

Operador ->

No sabíamos cómo usarlo correctamente para acceder a los miembros de un objeto a través de un puntero. Cometimos errores de sintaxis una y otra vez, y nos costó entender cuándo debía usar "->" en lugar de un punto ".".

Diseño de la relación entre las clases de los TAD implementados.

Explicación de los métodos más destacados.

Explicación de las funciones más importantes

MÉTODO 1

```
void processReservas(Cola* cola, PilaMesa* pila, Cola* cPedidos, ColaPedido* colapendientes, int *cantidadReservas, int *cantidadmesas);
```

La función **processReservas** procesa las reservas de una cola. Aquí está una explicación paso a paso:

1. Primero, extrae la primera reserva de la cola.
2. Crea una pila auxiliar y una variable booleana **encontrado** para rastrear si se ha encontrado una mesa adecuada.
3. Recorre la pila de mesas, eliminando cada mesa a su vez.
4. Para cada mesa eliminada, verifica si la mesa tiene suficiente capacidad y está en el mismo lugar que la reserva. Si es así, disminuye las variables **cantidadmesas** y **cantidadReservas**, imprime información sobre la reserva y la mesa, y marca **encontrado** como verdadero. Luego, crea un nuevo pedido con la mesa y la reserva, e inserta el pedido en la cola de pedidos pendientes.
5. Si la mesa no tiene suficiente capacidad o no está en el mismo lugar que la reserva, la mesa se inserta en la pila auxiliar.
6. Si la pila de mesas está vacía y no se ha encontrado ninguna mesa adecuada, la reserva se inserta de nuevo en la cola de reservas.
7. Finalmente, recorre la pila auxiliar, eliminando cada mesa a su vez e insertándola de nuevo en la pila de mesas.

MÉTODO 2

```
void processReservasHora (Cola* cola, PilaMesa* pila, Cola* cPedidos, ColaPedido *colapendientes, int *cantidadReservas, int *cantidadmesas);
```

La función **processReservasHora** procesa las reservas para una hora específica. Aquí está una explicación paso a paso:

1. Primero, pide al usuario que introduzca una hora (hora) para la cual quieren procesar las reservas.
2. Crea dos colas auxiliares: **cola_aux** y **cola_retorno**.
3. Luego recorre la cola principal de reservas (**cola**), eliminando cada reserva a su vez.
4. Para cada reserva eliminada, verifica si la hora de la reserva coincide con la hora especificada por el usuario. Si coincide, la reserva se inserta en **cola_aux** y se llama a **processReservas** para procesar las reservas en **cola_aux**.
5. Si la hora de la reserva no coincide con la hora especificada por el usuario, la reserva se inserta en **cola_retorno**.

- Después de que todas las reservas en la cola principal han sido revisadas, la función recorre **cola_retorno**, eliminando cada reserva a su vez e insertándola de nuevo en la cola principal.

MÉTODO 3

```
void processAllReservas(Cola* cola, PilaMesa* pila, Cola* pendientes, ColaPedido *colapedidos, int *cantidadReservas, int *cantidadmesas);
```

La función **processAllReservas** procesa todas las reservas en dos colas: **cola** y **pendientes**. Aquí está una explicación paso a paso:

- Inicializa una variable **numero** a 0. Esta variable se utiliza para controlar cuántas veces se ha llamado a la función **processReservas**.
- Entra en un bucle **while** que se ejecuta mientras haya reservas en **cola** o **pendientes**, y **numero** sea diferente de 30.
- Dentro del bucle, si **numero** es múltiplo de 3 y hay reservas en **pendientes**, llama a **processReservas** con **pendientes** como la cola de reservas a procesar.
- Si **numero** no es múltiplo de 3 y hay reservas en **cola**, llama a **processReservas** con **cola** como la cola de reservas a procesar.
- Incrementa **numero** en 1.

MÉTODO 4

```
void imprimirReservas(Cola* cola, PilaMesa* pila, Cola* cPedidos, ColaPedido *colapendientes, int *cantidadReservas, int *cantidadmesas);
```

La función **imprimirReservas** imprime la información sobre las reservas y las mesas. Aquí está una explicación paso a paso:

- Imprime **"Pila de mesas"** y luego llama a la función mostrar de la pila de mesas para imprimir la información sobre las mesas.
- Imprime **"Cola de reservas"** y luego llama a la función mostrar de la cola de reservas para imprimir la información sobre las reservas.
- Imprime **"Cola de pedidos"** y luego llama a la función mostrar de la cola de pedidos para imprimir la información sobre los pedidos.
- Imprime **"Cola de pendientes"** y luego llama a la función mostrar de la cola de pedidos pendientes para imprimir la información sobre los pedidos pendientes.
- Imprime **"Numero de mesas disponibles"** y luego imprime el número de mesas disponibles.

6. Imprime "**Reservas que quedan por resolver**" y luego imprime el número de reservas que quedan por resolver.

```
Reserva* Cola::eliminar()
```

La función **Cola::eliminar()** elimina el primer nodo de la cola y devuelve su valor. Aquí está una explicación paso a paso:

1. Crea un puntero nodo y lo apunta al primer nodo de la cola.
2. Si la cola está vacía (es decir, primero es **NULL**), retorna el valor de v (que no ha sido inicializado, lo cual puede ser un problema).
3. Si la cola no está vacía, actualiza el primer nodo de la cola para que sea el siguiente nodo en la cola.
4. Si después de esta actualización la cola está vacía (es decir, primero es **NULL**), establece ultimo a **NULL**.
5. Guarda el valor del nodo original (el que se está eliminando) en v.
6. Imprime un mensaje indicando que se está eliminando la reserva.
7. Elimina el nodo original de la memoria.
8. Retorna el valor del nodo eliminado.

```
void Cola::mostrar()
```

La función **Cola::mostrar()** imprime todos los elementos de la cola. Aquí está una explicación paso a paso:

1. Crea un puntero **aux** y lo apunta al primer nodo de la cola.
2. Imprime un encabezado para la lista de elementos de la cola.
3. Entra en un bucle **while** que se ejecuta mientras **aux** no sea **NULL**, es decir, mientras haya nodos en la cola.
4. Dentro del bucle, imprime los detalles de la reserva en el nodo actual. Estos detalles incluyen el nombre de la reserva, el número de personas, el lugar de la reserva, el menú y la hora. Para el nombre de la reserva, también calcula el tamaño del nombre y luego imprime espacios adicionales para asegurarse de que la línea tenga la longitud correcta.
5. Después de imprimir los detalles de la reserva, **aux** se actualiza para apuntar al siguiente nodo de la cola.
6. Finalmente, imprime una línea en blanco antes de pasar al siguiente nodo.

EXPLICACIÓN DEL COMPORTAMIENTO DEL PROGRAMA.

Inicio del programa:

```
C:\Users\isaac\Documents\Gi  X + v
-----
- Bienvenido al restaurante -
- 1. Generar aleatoriamente la cReservas -
- 2. Mostrar en pantalla los datos de la cReservas -
- 3. Borrar la cReservas -
- 4. Generar aleatoriamente la pMesas -
- 5. Mostrar en pantalla los datos de la pMesas -
- 6. Borrar la pMesas -
- 7. Realizar 1 pedido -
- 8. Simulacion la gestion 2 -
- 9. Simulacion la gestion 3 -
- 0. Salir -
-----
|
```

Opción 1

Al introducir el número 1, el programa ejecuta el case 1 y genera 12 reservas, 4 para cada hora. Y imprime por pantalla la cantidad.

```
1
+-----+
+ RESERVAS GENERADAS +
+-----+
CANTIDAD = 12
-----
- Bienvenido al restaurante -
- 1. Generar aleatoriamente la cReservas -
- 2. Mostrar en pantalla los datos de la cReservas -
- 3. Borrar la cReservas -
- 4. Generar aleatoriamente la pMesas -
- 5. Mostrar en pantalla los datos de la pMesas -
- 6. Borrar la pMesas -
- 7. Realizar 1 pedido -
- 8. Simulacion la gestion 2 -
- 9. Simulacion la gestion 3 -
- 0. Salir -
-----
|
```


Opción 2

Al introducir el número 2, el programa muestra las reservas creadas, las saca de la cola y las enseña.

```
#####  
#  
# Nombre de la reserva : Daniela #  
# Numero de personas : 5 #  
# Lugar de la reserva : interior #  
# Menu : completo #  
# Hora : 14:00 #  
#  
#####  
#  
# Nombre de la reserva : Luis #  
# Numero de personas : 5 #  
# Lugar de la reserva : interior #  
# Menu : vegano #  
# Hora : 15:00 #  
#  
#####  
#  
# Nombre de la reserva : Laura #  
# Numero de personas : 6 #  
# Lugar de la reserva : interior #  
# Menu : completo #  
# Hora : 15:00 #  
#  
#####  
#  
# Nombre de la reserva : Javier #  
# Numero de personas : 8 #  
# Lugar de la reserva : interior #  
# Menu : completo #  
# Hora : 15:00 #  
#  
#####  
#  
# Nombre de la reserva : Javier #  
# Numero de personas : 6 #  
# Lugar de la reserva : terraza #  
# Menu : vegano #  
# Hora : 15:00 #  
#  
#####
```

Opción 3

Al introducir el 3, borra las reservas y enseña por pantalla cuantas reservas quedan

```
eliminando la reserva de : Carlos  
eliminando la reserva de : Carlos  
eliminando la reserva de : Ana  
eliminando la reserva de : Ana  
eliminando la reserva de : Ana  
eliminando la reserva de : Laura  
eliminando la reserva de : Pablo  
eliminando la reserva de : Daniela  
eliminando la reserva de : Luis  
eliminando la reserva de : Laura  
eliminando la reserva de : Javier  
eliminando la reserva de : Javier  
Cantidad de reservas : 0  
=====
```

Opción 4

Al introducir el 4, genera las mesas 10 de 4 personas y 10 de 8 personas.

```
4

+++++
+ MESAS GENERADAS +
+++++

CANTIDAD = 20

-----
- Bienvenido al restaurante -
```

Opción 5

Al introducir el 5 muestra por pantalla las mesas creadas

```
C:\Users\isaac\Documents\Gr X +
#####
# Mesa: 5      #
# Capacidad: 4 #
# Lugar: interior #
#####
#
#####
# Mesa: 4      #
# Capacidad: 4 #
# Lugar: terraza #
#####
#
#####
# Mesa: 3      #
# Capacidad: 4 #
# Lugar: terraza #
#####
#
#####
# Mesa: 2      #
# Capacidad: 4 #
# Lugar: terraza #
#####
#
```

Opción 6

Al introducir 6 borra las mesas creadas.

```
-----
6
Cantidad de mesas : 0
-----
```

Opción 7

Al introducir el 7 genera un pedido con la primera reserva y lo muestra por pantalla, a su vez imprime que reservas quedan y que mesas quedan.

```
Cola de pedidos
+++++
+ Listado de todos los elementos de la cola: +
+++++
#####
#                                           #
# Nombre del cliente : Carlos             #
# Numero de personas : 1                 #
# Numero de mesa : # Mesa: 10            #
# menu : completo                        #
# hora : 13:00                           #
# PEDIDO SIN FINALIZAR                    #
#####

Cola de pendientes
+++++
+ Listado de todos los elementos de la cola: +
+++++
Numero de mesas disponibles
19
Reservas que quedan por resolver
11
=====
```

Opción 8

Al introducir 8, te pregunta que reservas quieres realizar, hay que introducir una fecha,

0 para la 13:00 , 1 para las 14:00 y 2 para las 15:00

```
-----
8
dime la hora de las reservas que quieres realizar : 32759
introduce 0 para las 13:00, 1 para las 14:00 y 2 para las 15:00
|
```

Una vez introducida la hora, solo realiza esas reservas y crea los pedidos

```
+++++
+ Listado de todos los elementos de la cola: +
+++++
#####
#                                           #
# Nombre del cliente : Carlos             #
# Numero de personas : 1                 #
# Numero de mesa : # Mesa: 10            #
# menu : completo                        #
# hora : 13:00                           #
# PEDIDO SIN FINALIZAR                    #
#####

#####
#                                           #
# Nombre del cliente : Carlos             #
# Numero de personas : 2                 #
# Numero de mesa : # Mesa: 7            #
# menu : sin gluten                      #
# hora : 13:00                           #
# PEDIDO SIN FINALIZAR                    #
#####

#####
#                                           #
# Nombre del cliente : Ana                #
# Numero de personas : 8                 #
# Numero de mesa : # Mesa: 20            #
# menu : vegano                          #
# hora : 13:00                           #
# PEDIDO SIN FINALIZAR                    #
#####

#####
#                                           #
# Nombre del cliente : Carlos             #
# Numero de personas : 4                 #
# Numero de mesa : # Mesa: 9            #
# menu : sin gluten                      #
# hora : 13:00                           #
# PEDIDO SIN FINALIZAR                    #
#####
```

Opción 9

La opción 9 realiza todos los pedidos, incluido los que estén en pendientes, esto se puede observar si generas mas mesas, los pedidos pendientes también los introduce para realizar los pedidos,

```
#####
#
# Nombre del cliente : Carlos      #
# Numero de personas : 1          #
# Numero de mesa : # Mesa: 8      #
# menu : vegano                   #
# hora : 15:00                    #
# PEDIDO SIN FINALIZAR            #
#####

#####
#
# Nombre del cliente : Javier      #
# Numero de personas : 5          #
# Numero de mesa : # Mesa: 17     #
# menu : vegano                   #
# hora : 15:00                    #
# PEDIDO SIN FINALIZAR            #
#####

#####
#
# Nombre del cliente : Anabel      #
# Numero de personas : 3          #
# Numero de mesa : # Mesa: 4      #
# menu : completo                 #
# hora : 15:00                    #
# PEDIDO SIN FINALIZAR            #
#####

#####
#
# Nombre del cliente : Oscar       #
# Numero de personas : 3          #
# Numero de mesa : # Mesa: 3      #
# menu : vegano                   #
# hora : 15:00                    #
# PEDIDO SIN FINALIZAR            #
#####
```

Opción 0

Salir del programa, borra las estructuras creadas con delete.

Objetos:

Mesa: la clase Mesa representa mesas de restaurante con números, capacidad y ubicación. Está relacionada con PilaMesa. Puedes crear mesas por defecto o personalizadas y eliminarlas. Ofrece métodos para mostrar y generar mesas aleatorias. También incluye funciones para acceder y modificar sus atributos.

Reservas: la clase Reserva guarda información detallada de reservas en un restaurante, incluyendo nombre, ubicación, número de personas, hora y menú. Es amiga de la clase Cola para compartir datos. Permite crear reservas personalizadas o predeterminadas, así como eliminarlas. Tiene métodos para escribir, mostrar y generar reservas aleatorias. Además, provee funciones para acceder y modificar datos de la reserva, como el nombre, número de personas y más.

Pedido: la clase Pedido representa un formulario de pedido en un restaurante, guardando información sobre la mesa, el cliente, el menú, la ubicación, la hora y el estado del pedido. Está relacionada con ColaPedido. Ofrece opciones para crear pedidos predeterminados o personalizados, así como para borrarlos. Puede generar un pedido a partir de una mesa y una reserva. Proporciona métodos para acceder y modificar los datos del formulario.

Bibliografía

Hemos usado los archivos subidos a la blackboard como guía.



ejemplo_pila

Archivos adjuntos: [Pilas ejemplo.pdf](#) (43,859 KB)



Ejemplo Cola

Archivos adjuntos: [Colas ejemplo.pdf](#) (32,451 KB)



CLASE 16 OCTUBRE

Archivos adjuntos: [Ej3_clase_clientes.rar](#) (359,946 KB)
 [colas_sep.rar](#) (331,958 KB)
 [pilas_sep.rar](#) (332,408 KB)

PROYECTOS VISTOS EN LA CLASE DEL 16 DE OCTUBRE